УТВЕРЖДЕН
RU.17701729.12.10-01 ПЗ 01-1-ЛУ

УМНАЯ КОММУНИКАЦИОННАЯ ПЛАТФОРМА

Пояснительная записка

RU.17701729.12.10-01 ПЗ 01-1

Листов 18

Подп. и дата	
Инв. № дубл.	
Взам. инв. №	
Подп. и дата	
Инв. № подл	

Москва 2026

1. ВВЕДЕНИЕ

1.1. Наименование программы

Наименование программы – «Умная коммуникационная платформа».

Краткое наименование – «SmartCom Platform».

1.2. Документы, на основании которых ведется разработка

Учебный план подготовки бакалавров по направлению 09.03.04 «Программная инженерия» НИУ ВШЭ и утвержденная академическим руководителем тема выбранного курсового проекта.

2. НАЗНАЧЕНИЕ И ОБЛАСТЬ ПРИМЕНЕНИЯ

2.1. Назначение программы

2.1.1. Функциональное назначение

Программа обеспечивает:

1. Модульную интеграцию с внешними провайдерами связи для приема/совершения отправки сообщений и звонков.
2. Модульную интеграцию с внешними AI-системами.
3. Веб-интерфейс для операторов для взаимодействия с:
   - системой управления контактами, включающая работу с карточкой клиента, историей взаимодействий, заметок по клиенту и задачами по нему.
   - системой навигации и уведомлений по текущим задачам оператора.
   - системой группировки клиентов и операторов для разделения полномочий операторов в рамках одной организации.
4. Автономную работу всех модулей, так система устойчива к внешним и внутренним падениям.

2.1.2. Эксплуатационное назначение

Программа предназначена для использования в бизнес-среде для повышения надежности, доступности и эффективности клиентских коммуникаций. Позволяет сократить время обработки звонков, стандартизировать рабочее место оператора, устранить зависимость от сбоев CRM и централизовать управление коммуникациями. Благодаря выносу узла оркестратора связи из CRM во внешнюю систему — нашу платформу (см. Приложение 1), и проектирования системы в виде модулей: все есть внешняя зависимость, подключение к которой производит пользователь через предоставляемый интерфейс. На сравнительной схеме в Приложении 1 показано, как узел связи переносится из монолитной CRM-зависимой архитектуры в предлагаемую модульную платформу, где наша система выступает оркестратором коммуникаций.

2.2. Краткая характеристика и область применения программы

«SmartCom Platform» – это облачная CPaaS (Communication Platform as a Service) система, предназначенная для бизнесов, взаимодействующих с клиентами посредством телефонной связи и связи посредством интернета, таких как электронная почта и мессенджеры (службы поддержки, колл-центры, отделы продаж).

Платформа выступает автономной универсальным клиентом к провайдерам связи, предоставляя операторам единый веб-интерфейс для работы с сообщениями, со звонками, историей

взаимодействий и клиентскими данными. Ключевые преимущества: независимость от внешних CRM, единое рабочее пространство, модульность интеграций, AI-ассистенты, оптимизирующие работу оператора.
Область применения: малый и средний бизнес, а также подразделения крупных компаний, где критична надежность телефонных коммуникаций и эффективность операторов.

3. ТЕХНИЧЕСКИЕ ХАРАКТЕРИСТИКИ

3.1. Постановка задачи на разработку программы

Программа обеспечивает автономную работу операторов с телефонными и текстовыми коммуникациями через различные каналы связи, предоставляет единый веб-интерфейс для управления контактами и историей взаимодействий, а также содержит AI-ассистентов, которые в реальном времени исправляют ошибки в сообщениях и предлагают создание задач на основе анализа переписки.

3.2. Описание применяемых методов (алгоритмические решения)

3.2.1. Защита страниц от несанкционированного доступа на клиентской стороне

Для предотвращения кратковременного отображения содержимого страницы пользователю, не прошедшему аутентификацию (эффект «моргания»), на клиентской части реализован механизм предварительной проверки прав доступа. Служебные скрипты (protect.js) и стили (protect.css) скрывают основное содержимое страницы до тех пор, пока не будет получен ответ от сервера, подтверждающий валидность сессии. В случае отрицательного ответа сразу отображается страница аутентификации без промежуточной прорисовки защищённых элементов. Это улучшает пользовательский опыт и исключает утечку статической информации интерфейса неавторизованным лицам.

3.2.2. Поддержка горячих клавиш для ускорения работы оператора

В интерфейсе платформы реализована расширенная обработка горячих клавиш. Нажатие Ctrl + Enter инициирует отправку сообщения, Alt + I открывает поле ввода нового сообщения и т.д. Такая механика сокращает количество кликов и позволяет оператору выполнять частые действия без переключения на мышь, что повышает скорость работы и снижает когнитивную нагрузку в процессе интенсивных коммуникаций.

3.2.3. Унификация взаимодействия с сервером и идемпотентность запросов

Все HTTP-запросы к backend API централизованы в модуле api.js, построенном на базе библиотеки Axios. Это позволяет единообразно настраивать заголовки, обрабатывать ошибки и внедрять сквозную функциональность. Важным элементом модуля является генерация и хранение токенов идемпотентности для критичных операций (в первую очередь — отправка сообщений). Токен идемпотентности создаётся на клиенте до выполнения запроса и передаётся в заголовке. Обновление токена происходит только после получения подтверждения от сервера о том, что асинхронная задача на отправку создана; кнопки отправки при этом остаются заблокированными до момента смены токена. Таким образом, при любых повторных попытках отправки (из-за сетевых сбоев или дублирования действий пользователем) сервер способен распознать дубликат и не допустить повторной обработки, обеспечивая семантику «exactly-once».

3.2.4. Механизм аутентификации

Взаимодействие фронтенда и бекенда защищено при помощи JWT (JSON Web Token). После успешной проверки учётных данных сервер выпускает подписанный токен, который клиент передаёт при каждом последующем запросе. Проверка токена выполняется без обращения к базе данных — исключительно по криптографической подписи, что минимизирует задержки и снижает нагрузку на хранилище. Токен имеет ограниченный срок жизни и может быть обновлён через механизм refresh-токена.

3.2.5. Алгоритм отправки сообщения через модуль связи

Процесс отправки сообщения разработан с учётом требований надёжности и гарантированной доставки (см. Приложение 2). Ниже приведено описание основных шагов (на примере исходящего сообщения; входящие сообщения обрабатываются по аналогичной схеме с точностью до направления потока данных):

1. **Инициирование отправки.** Пользователь вводит текст сообщения и нажимает «Отправить» (или горячую клавишу). Фронтенд генерирует токен идемпотентности, вызывает POST /send/message с телом сообщения и токеном.

2. **Аутентификация и валидация.** Backend API проверяет JWT-токен. При неудаче возвращается ошибка 401 Unauthorized, и процесс прерывается.

3. **Фиксация в Outbox.** При успешной аутентификации backend записывает сообщение в специальную таблицу outbox в PostgreSQL с начальным статусом pending. Затем API немедленно возвращает ответ 201 Created фронтенду, который может разблокировать интерфейс.

4. **Асинхронная вычитка Outbox Worker'ом.** Специальный фоновый процесс (Outbox Worker) с заданной периодичностью выбирает из таблицы записи со статусом pending, переводит их в processing и помещает задание в очередь сообщений (Message Broker).

5. **Обработка отправки (Sender Worker).** Другой процесс-обработчик (Sender Worker) читает задания из очереди. Для каждого сообщения он:

   - обращается к базе данных контактов и источников связи (Contacts, Origins PG), чтобы проверить принадлежность номера/идентификатора. На основе этих данных, получить из Local Storage конкретную реализацию интерфейса сендера (к внешнему провайдеру).

   - вызывает метод отправки у соответствующего провайдера.

   - по результату обновляет статус записи в outbox на sent или failed, а также изменяет статус задания в очереди.

6. **Изоляция провайдеров.** Реализация взаимодействия с каждым провайдером (Telegram, SMS Aero и др.) скрыта за унифицированным интерфейсом. Добавление нового канала сводится к созданию класса, реализующего этот интерфейс, и регистрации его в локальном реестре (Local Storage конфигурации). Ошибки в работе одного провайдера не затрагивают

остальные.

3.2.6. Алгоритм исправления текста с помощью AI-ассистента

AI-подсистема также построена на асинхронном взаимодействии и демонстрирует унифицированный подход к интеграции внешних сервисов. Диаграмма последовательности для сценария «Исправление сообщения» приведена в Приложении 3 (см. Приложение 3). По аналогичному принципу работает и второй ассистент («Создание задачи»), с той лишь разницей, что на вход подаётся история сообщений, а на выходе — предложение задачи.

Шаги алгоритма исправления текста:

1. **Отправка черновика.** Оператор вводит текст и активирует функцию «Исправить». Фронтенд через WebSocket-соединение отправляет черновик на backend.
2. **Получение конфигурации ассистента.** Backend запрашивает из базы данных настройки AI-агента для текущего пользователя (URL, таймаут, инструкцию — промпт). Эти настройки задаются через интерфейс платформы.
3. **Создание асинхронной задачи.** Backend генерирует уникальный ключ сообщения и отправляет запрос к API внешнего AI-сервиса (в базовой реализации — Yandex Cloud AI Studio), передавая текст сообщения и инструкцию. Сервис возвращает идентификатор задачи.
4. **Ожидание результата с адаптивным поллингом.** Пользователь в настройках AI-агента может указать максимальное время ожидания (дедлайн). Если ответ от внешнего сервиса не получен сразу, backend начинает циклический опрос состояния задачи. Интервал опроса динамически увеличивается: первый запрос отправляется через 1 секунду, второй — через 2 секунды, третий — через 3 и так далее (линейный рост). Это позволяет, с одной стороны, быстро реагировать на завершение коротких задач (исправление текста обычно занимает мало времени), а с другой — не перегружать сеть и внешний сервис при длительном ожидании. Клиентский интерфейс в это время не блокируется.
5. **Получение и возврат исправленного текста.** Когда задача завершена, backend получает результат, сохраняет его во временное хранилище с ключом сообщения, закрывает WebSocket-сессию и отправляет результат фронтенду. Фронтенд отображает пользователю исправленный текст с выделением изменений. Если за отведённый дедлайн ответ не получен, оператору направляется уведомление о превышении времени, и он может повторить запрос или продолжить без исправлений.
6. **Гибкость замены ИИ-модели.** Благодаря тому, что URL, промпт и таймаут хранятся в базе данных и могут редактироваться в разделе «Интеграции», пользователь способен без изменения кода платформы переключиться на другой AI-сервис, реализующий тот же REST-контракт. Стандартные ассистенты поставляются как преднастроенные шаблоны.

3.2.7. Архитектура базы данных: модель хранения контактов и история идентификаторов

Гибкость и расширяемость контактной базы обеспечивается нормализованной схемой данных, ключевой особенностью которой является таблица соединений (connections). Она хранит информацию контактов (contacts) о идентификаторах в системах связи (см. Приложение 4), позволяя предоставлять через них коммуникацию.
Основные принципы:

1. **Добавление новых каналов без миграции схемы.** Появление нового провайдера связи не требует изменения структуры основных таблиц. Достаточно добавить запись в таблицу-справочник источников (channel_type, channel_categories), реализовать соответствующий интерфейс отправки/приёма и зарегистрировать его в конфигурации (Local Storage).
2. **Поддержка историчности и предотвращение ошибочных коммуникаций.** Каждое соединение имеет признак активности. При смене владельца телефонного номера или другого идентификатора старый connection деактивируется, а для нового контакта создаётся новая активная запись. Это позволяет:
   - всегда видеть историю сообщений и звонков по конкретному идентификатору, даже если он больше не принадлежит текущему контакту;
   - исключить отправку сообщений на устаревшие или чужие реквизиты;
   - при повторной активации номера у прежнего контакта восстанавливать связь без потери архивных данных.

Такая модель данных обеспечивает выполнение требования гибкости, сформулированного в техническом задании: добавление новых полей, параметров и метрик не требует изменения основной структуры, а новая функциональность наращивается через конфигурацию, а не через рефакторинг ядра.

3.2.8. Контрактное программирование на основе OpenAPI

Для исключения ошибок несоответствия запросов и ответов между клиентской и серверной частями в проекте применён подход «Contract First». Интерфейсы всех HTTP-ручек описаны в спецификации OpenAPI 3.0.0. На основе этой спецификации автоматически генерируются DTO-структуры на языке C++ для серверной части;

Такой подход гарантирует, что любые расхождения в типах данных, именах полей или сигнатурах методов выявляются на этапе компиляции/сборки, а не в рантайме. Это существенно повышает стабильность взаимодействия между слоями системы и упрощает рефакторинг: при изменении API достаточно обновить спецификацию и перегенерировать код, после чего компилятор укажет все места, требующие корректировки.

3.3. Описание и обоснование выбора метода организации входных и выходных данных

1. Входные данные:
   - События от провайдеров телефонии (входящие сообщения и звонки).
   - Данные от внешних AI-систем (исправленные сообщения, предложения по созданию задач).
   - Действия операторов в веб-интерфейсе (нажатия кнопок, ввод текста).
   - Команды настройки (интеграций, организации, операторов).
2. **Выходные данные**:
   - События к провайдерам телефонии (исходящие сообщения и звонки).
   - Данные, отправляемые во внешние AI-системы (сообщения операторов, которые они хотят улучшить, сообщения переписки).
   - Графический интерфейс.
   - Информация о собеседниках (операторы, контакты) и история коммуникаций.
   - Пользовательские настройки.
   - Логи событий.

Выбор протоколов продиктован требованиями к безопасности, скорости и совместимости.

- **HTTPS** применяется для всех REST-вызовов между фронтендом и бекендом, а также при обращении к API провайдеров связи и AI-сервисов. Обеспечивает шифрование трафика, аутентификацию сервера и целостность данных, что критично при передаче персональной информации клиентов и содержимого переписки.
- **WebSocket** используется для событийно-ориентированного взаимодействия в реальном времени: уведомления о входящих звонках, мгновенное отображение новых сообщений, обновление статусов AI-задач. В отличие от периодического опроса (polling), постоянное двустороннее соединение минимизирует задержки (менее 1 секунды) и снижает накладные расходы на установку многочисленных HTTP-соединений.
- **SMTP (Simple Mail Transfer Protocol)** задействован при отправке электронных писем с домена пользователя. Письма передаются напрямую на SMTP-сервер получателя или через ретранслятор, что гарантирует доставку в соответствии со стандартами электронной почты.
- **IMAP (Internet Message Access Protocol)** используется для приёма входящей корреспонденции, когда пользователь подключает собственный почтовый ящик.

Платформа периодически опрашивает IMAP-сервер, забирает новые письма и отображает их в интерфейсе оператора как ещё один канал коммуникации. Такой подход обеспечивает совместимость с любыми почтовыми провайдерами без необходимости развёртывания собственного почтового сервера.

3.4. Описание допущений и ограничений

3.4.1. Допущения

1. Стабильность внешних API. Предполагается, что API провайдеров связи и AI-сервисов функционируют в соответствии с опубликованной документацией, а временные сбои носят краткосрочный характер и компенсируются механизмами повторных попыток и очередями.
2. Корректность конфигурации. Администратор системы корректно вводит учётные данные, ключи доступа и URL-адреса внешних сервисов; валидация этих данных в реальном времени не выполняется (проверяется только факт подключения).
3. Квалификация пользователей. Пользователи обладают базовыми навыками работы с веб-интерфейсами; специальное обучение работе с платформой не требуется.
4. Ограниченная нагрузка. Система проектируется под нужды малого и среднего бизнеса; пиковые нагрузки соответствуют одновременной работе нескольких десятков операторов в рамках одной организации.

3.4.2. Ограничения

1. Зависимость от внешних сервисов:
   - Если провайдер связи изменяет API без обратной совместимости, потребуется доработка соответствующего адаптера.
   - Качество исправления текста и релевантность предложенных задач напрямую зависят от используемой AI-модели; платформа не гарантирует безошибочность AI-рекомендаций.
2. Отправка SMS и совершение звонков ограничены тарифами и лимитами, установленными провайдерами на стороне клиента.
3. Для стабильной работы необходим современный браузер с поддержкой WebSocket и JavaScript. Минимальное разрешение экрана – 1280×720 пикселей.

4. ОЖИДАЕМЫЕ ТЕХНИКО-ЭКОНОМИЧЕСКИЕ ПОКАЗАТЕЛИ

Внедрение разработанной коммуникационной платформы позволяет бизнесу получить следующие операционные и экономические преимущества по сравнению с традиционной схемой «телефония внутри CRM» и разрозненными каналами связи.

Оперативные показатели (ожидаемые):

1. Сокращение среднего времени обработки одного клиента на 20–30 %.

   Достигается за счёт единого веб-интерфейса оператора, объединяющего все каналы связи, карточку клиента, историю взаимодействий и задачи. Исключается необходимость переключения между несколькими приложениями.

2. Снижение времени простоя операторов при сбоях внешних CRM до нуля.

   Благодаря автономной архитектуре платформы (локальное хранение контактов и истории) оператор продолжает принимать и совершать звонки, даже если внешняя CRM недоступна.

3. Повышение производительности труда операторов за счёт AI-ассистентов.

   Функция автоматического исправления орфографических и пунктуационных ошибок сокращает время на вычитку сообщений. Ассистент создания задач снижает когнитивную нагрузку, избавляя оператора от необходимости помнить о каждом обещании клиенту.

4. Уменьшение затрат на обучение новых сотрудников.

   Унифицированный интерфейс с поддержкой горячих клавиш и настраиваемыми панелями позволяет оператору быстрее освоить систему по сравнению с пакетом разрозненных инструментов.

Экономические показатели:

1. Снижение зависимости от конкретного вендора телефонии.

   Модульная архитектура адаптеров позволяет подключать новых провайдеров без перестройки бизнес-процессов. Это даёт возможность выбирать наиболее выгодные тарифы и избегать блокировки при изменении условий текущим оператором связи.

2. Потенциальное увеличение объёма обрабатываемых заявок без расширения штата.

   При сокращении времени обработки одного клиента существующая команда операторов может обслужить пропорционально большее количество обращений, что прямо влияет на выручку компании.

3. Конкурентное преимущество для малого и среднего бизнеса.

   Платформа предоставляет функциональность, ранее доступную только крупным

компаниям с дорогими CRM-решениями: омниканальность, AI-подсказки, надёжное хранение истории.

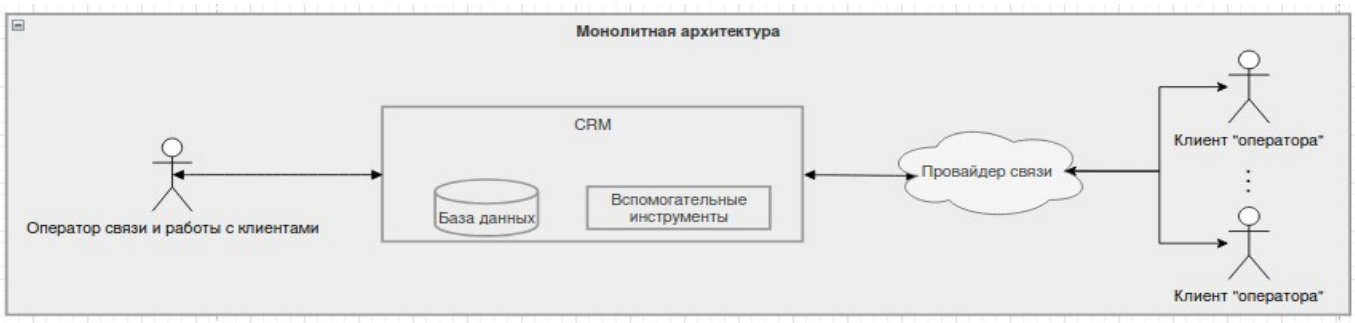
Таким образом, выбранный вариант технического решения (автономная модульная платформа с AI-ассистентами) обеспечивает ощутимые оперативные и экономические выгоды по сравнению с традиционными CRM-зависимыми системами.

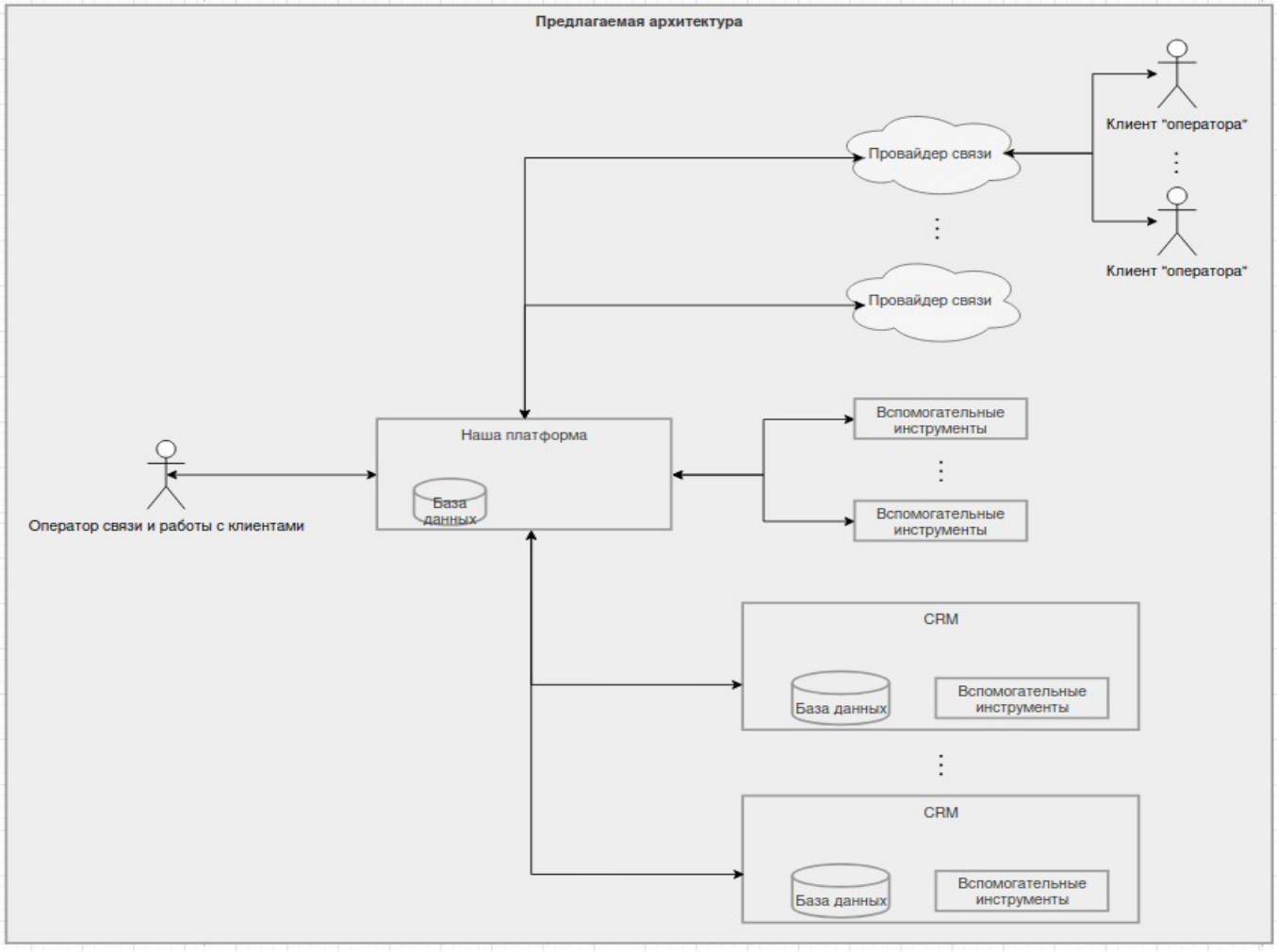
5. ИСТОЧНИКИ, ИСПОЛЬЗОВАННЫЕ ПРИ РАЗРАБОТКЕ

- Техническое задание (ГОСТ 19.201-78).
- Пояснительная записка (ГОСТ 19.404-79).
- Руководство оператора (ГОСТ 19.505-79).
- Программа и методика испытаний (ГОСТ 19.301-79).
- Текст программы (ГОСТ 19.401-78).
- userver – документация фреймворка для разработки высоконагруженных серверных приложений на C++.
- OpenAPI Specification 3.0.0 – спецификация для описания интерфейсов REST API.
- CMake – система сборки, документация и руководства по организации кросс-платформенной сборки проекта.
- Development Containers (CLI) – спецификация и инструментарий для создания изолированных сред разработки.
- Yandex Cloud AI Studio – документация и API для создания и использования генеративных моделей.
- MTC Exolve – документация API провайдера связи (голос, SMS).
- SMS Aero – документация API сервиса рассылки SMS и WhatsApp-сообщений.

Приложение 1

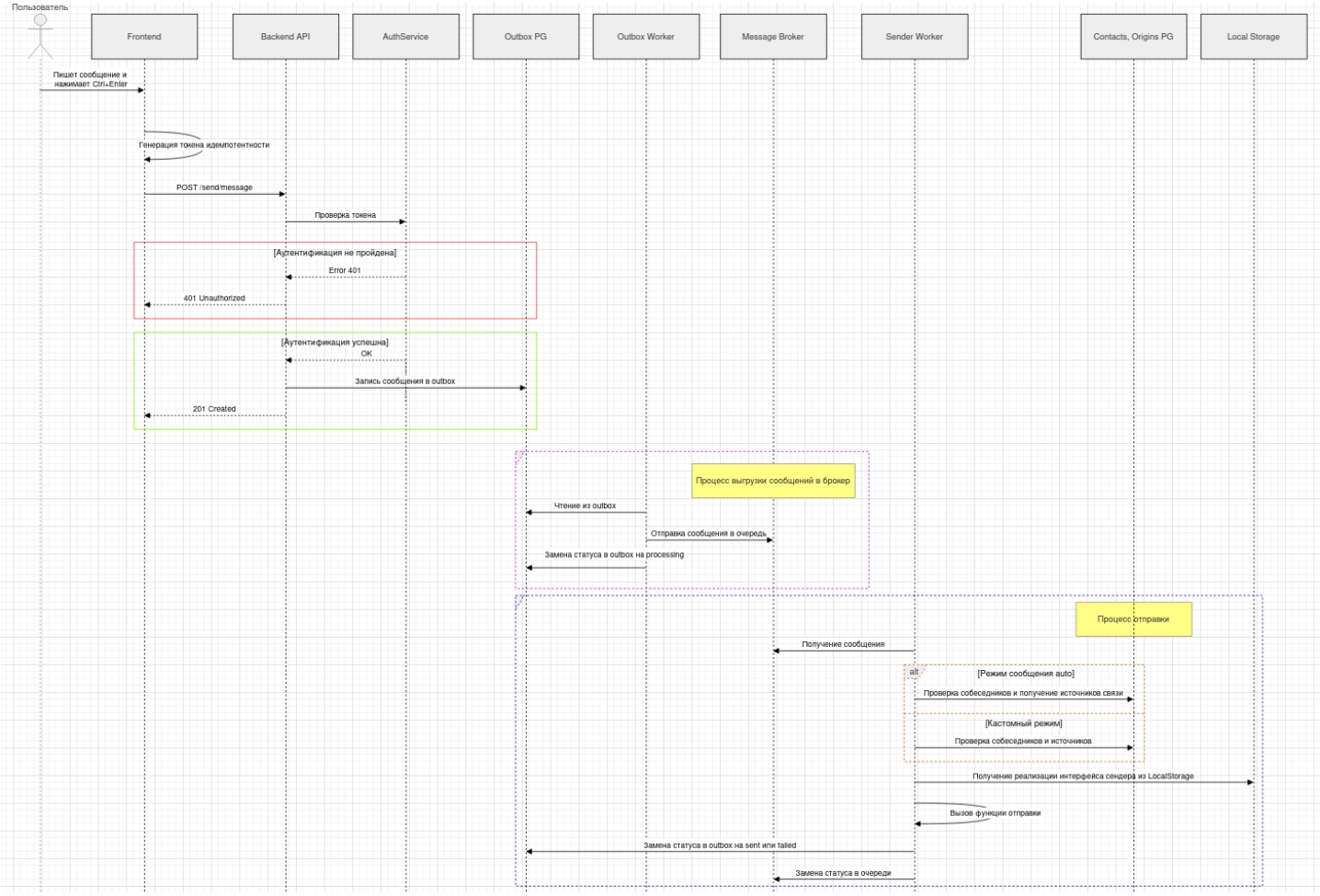
Архитектура приложения с сравнением от «монолитной» CRM-зависимости к модульной платформе





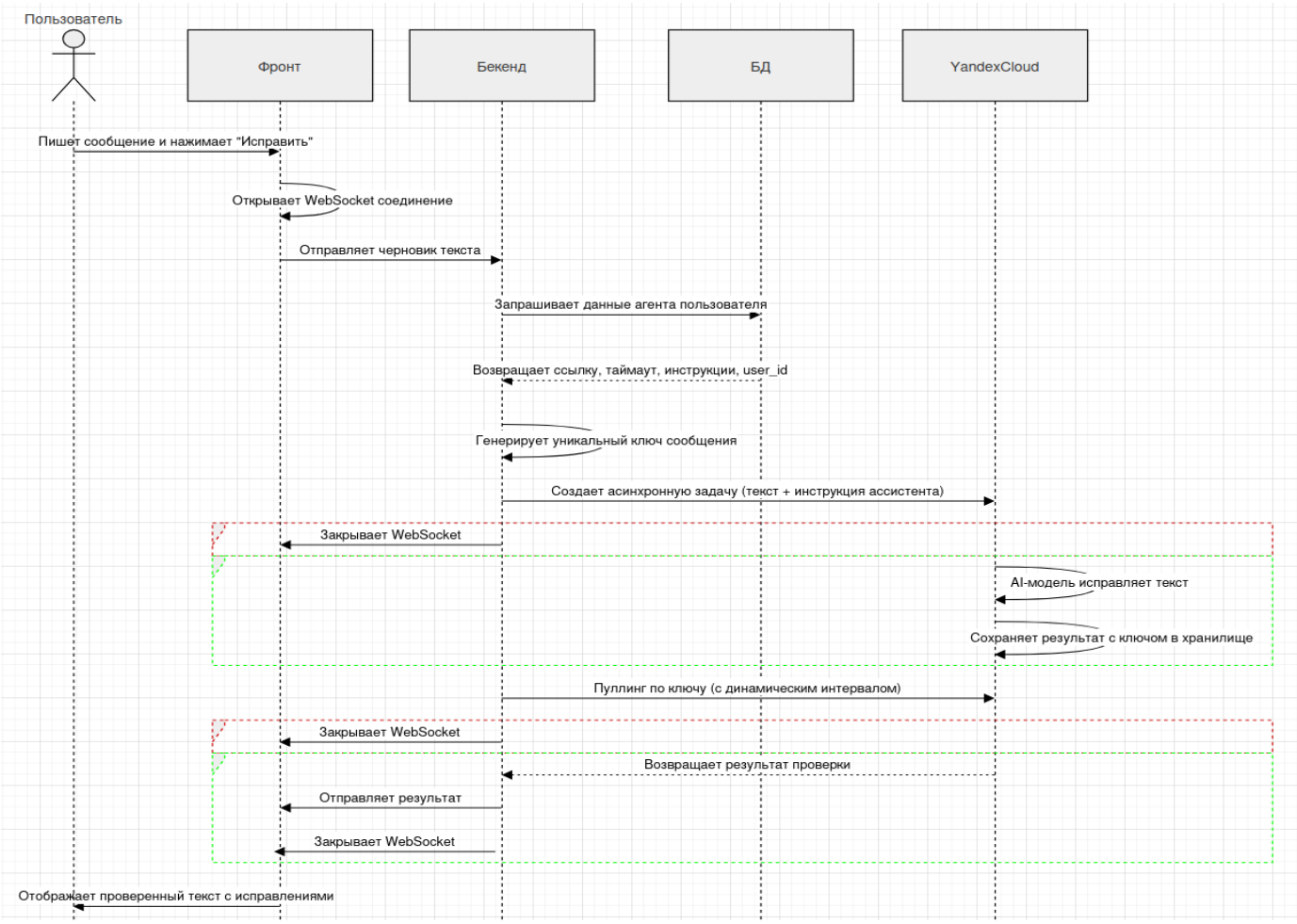
Приложение 2

Схема запроса на отправку сообщения



Приложение 3

Схема запроса на исправление сообщения AI-ассистентом



Приложение 4

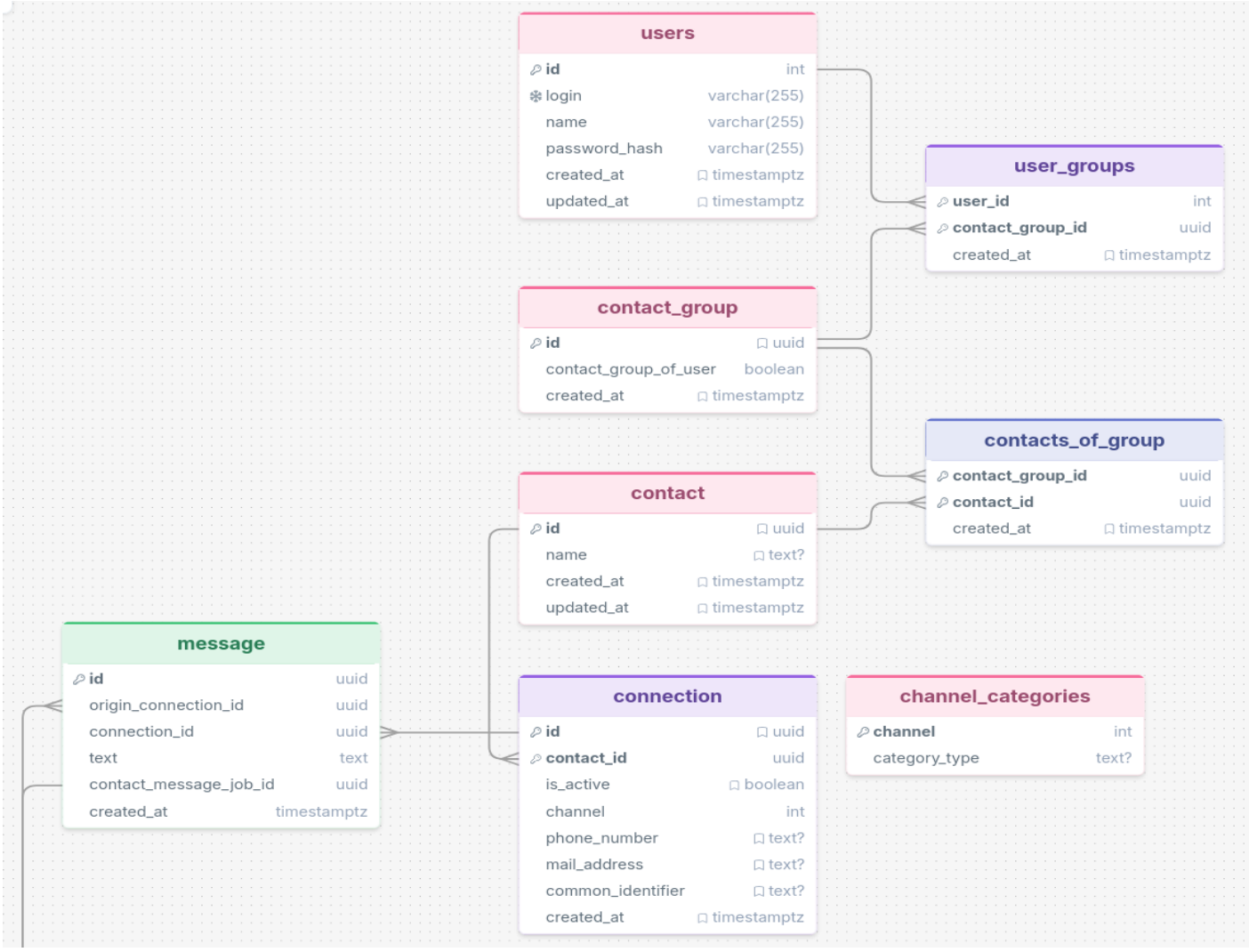



Схема части базы данных, связанной с коннекшенами контакта